Week 4 - Lesson 2
Plug-and-Play Sentiment with BERT-Family Pipelines

Create your own notebook | Approximately 45 minutes, excluding downloads

You will create a Python notebook, evaluate sentiment on 500 balanced Amazon reviews, compute accuracy and F1, and reflect on model quality, processing time, and memory. **Copy each code block below into its own code cell, in order.** You will write the final reflection yourself.

1. Create a blank notebook in Google Colab

Open Google Colab and select **File > New notebook**. Rename it **bert_sentiment_pipeline.ipynb**. GPU is optional: use **Runtime > Change runtime type** if you wish. Add a markdown title, then create the code cells below.

2. Install packages and import libraries

Run this cell first. If prompted to restart after installation, restart and rerun the cells from the top. Public model and dataset downloads require internet access, but no paid API or token is needed.

```
%pip install -q "transformers==4.57.6" "datasets>=3,<6" \
    "evaluate==0.4.6" "torch>=2.6" scikit-learn

import os
os.environ["USE_TF"] = "0"
os.environ["USE_FLAX"] = "0"
import time
import torch
import pandas as pd
import evaluate
from transformers import pipeline
from datasets import load_dataset

SEED, BATCH_SIZE, MAX_LENGTH = 42, 8, 256
DEVICE = 0 if torch.cuda.is_available() else -1
torch.set_num_threads(2)
print("Device:", "GPU" if DEVICE == 0 else "CPU")
```

3. Load 500 balanced Amazon reviews

The dataset labels already mean **0 = negative and 1 = positive**; keep them unchanged. Sample 250 of each class from the first 10,000 test records after removing repeated review bodies. This is a limited teaching sample, not the full benchmark. Do not train or tune thresholds on these evaluation reviews.

```
stream = load_dataset(
    "fancyzhx/amazon_polarity", split="test", streaming=True
)
assert stream.features["label"].names == ["negative", "positive"]
pool = pd.DataFrame(list(stream.take(10000)))
pool = pool.drop_duplicates(subset="content")
parts = [pool[pool["label"] == y].sample(n=250, random_state=SEED)
         for y in (0, 1)]
sample = pd.concat(parts).sample(frac=1, random_state=SEED)
sample = sample.reset_index(drop=True)
texts = sample["content"].tolist()
labels = sample["label"].astype(int).tolist()
assert len(texts) == 500 and sample["content"].is_unique
assert sample["label"].value_counts().to_dict() == {0: 250, 1: 250}
print("Class counts:", sample["label"].value_counts().sort_index().to_dict())
```

4. Instantiate the sentiment pipeline

Use this **sentiment-fine-tuned DistilBERT checkpoint**, which belongs to the BERT family. A bare pretrained BERT model is not a ready-made sentiment classifier. Keep the original review text and let the checkpoint's own tokenizer process it. Parameter storage is reported separately; it is not total or peak RAM.

```
MODEL_ID = ("distilbert/"
            "distilbert-base-uncased-finetuned-sst-2-english")
clf = pipeline("sentiment-analysis", model=MODEL_ID,
               framework="pt", device=DEVICE, dtype=torch.float32)
print("Checkpoint:", MODEL_ID)
print("Model labels:", clf.model.config.id2label)

def parameter_mib(classifier):
    return sum(p.numel() * p.element_size()
               for p in classifier.model.parameters()) / (1024 ** 2)

print("Parameter storage (MiB):", round(parameter_mib(clf), 1))
```

5. Run batch inference and measure processing time

Use batch size 8 and explicitly truncate at **256 tokenizer tokens, including special tokens**. The first call warms up the model. The timed call includes tokenization, inference, and label post-processing, but excludes downloading and model loading. CUDA is synchronized when used.

```
def synchronize():
    if DEVICE == 0:
        torch.cuda.synchronize()

def predict_and_time(classifier):
    kwargs = dict(batch_size=BATCH_SIZE, truncation=True,
                  max_length=MAX_LENGTH, num_workers=0)
    classifier(texts[:BATCH_SIZE], **kwargs)  # Warm-up; not timed.
    synchronize()
    start = time.perf_counter()
    outputs = classifier(texts, **kwargs)
    synchronize()
    elapsed = time.perf_counter() - start
    assert len(outputs) == len(texts)
    return outputs, elapsed

token_ids = clf.tokenizer(texts, truncation=False, verbose=False)["input_ids"]
print("Truncated reviews:", sum(len(x) > MAX_LENGTH for x in token_ids))
outputs, elapsed = predict_and_time(clf)
print(f"Time for 500 reviews: {elapsed:.2f} s")
print(f"Throughput: {len(texts) / elapsed:.2f} reviews/s")
print(f"Amortized time: {1000 * elapsed / len(texts):.2f} ms/review")
```

Interpret the timing correctly: the printed ms/review is amortized batched processing time, not the latency of one standalone request. Record the device, batch size, and token cap. Truncation may remove a review's conclusion; note how many reviews exceed the cap.

6. Compute accuracy and positive-class F1

Convert the pipeline's string labels to integer predictions. Accuracy is the fraction correct. F1 combines precision and recall for **positive label 1**. Scores use a 0-1 scale. Inspect up to three errors; if the text is shortened, read the full review before interpreting it.

```
label_map = {"NEGATIVE": 0, "POSITIVE": 1}
preds = [label_map[p["label"]] for p in outputs]
accuracy_metric = evaluate.load("accuracy")
f1_metric = evaluate.load("f1")
accuracy = accuracy_metric.compute(predictions=preds, references=labels)
f1 = f1_metric.compute(predictions=preds, references=labels,
                       average="binary", pos_label=1)
print("Accuracy:", accuracy["accuracy"])
print("Positive-class F1:", f1["f1"])

wrong = [i for i, (p, y) in enumerate(zip(preds, labels)) if p != y]
print("Misclassified reviews:", len(wrong))
for i in wrong[:3]:
    print("\nGold:", labels[i], "Predicted:", preds[i])
    print(texts[i][:400])
```

7. Optional: try a second sentiment-fine-tuned checkpoint

Run the next cell only if you want a comparison. It downloads a BERT-base checkpoint fine-tuned on SST-2 and uses its explicit label mapping. Use the same 500 reviews, device, batch size, and token limit. Verify label meanings before substituting another checkpoint.

```
# Optional: skip this cell if you only evaluate the default model.
second_id = "textattack/bert-base-uncased-SST-2"
second = pipeline("sentiment-analysis", model=second_id,
                  framework="pt", device=DEVICE, dtype=torch.float32)
out2, elapsed2 = predict_and_time(second)
second_map = {"LABEL_0": 0, "LABEL_1": 1}
preds2 = [second_map[p["label"]] for p in out2]
print("Checkpoint:", second_id)
print(accuracy_metric.compute(predictions=preds2, references=labels))
print(f1_metric.compute(predictions=preds2, references=labels,
                        average="binary", pos_label=1))
print(f"Time for 500 reviews: {elapsed2:.2f} s")
print(f"Amortized time: {1000 * elapsed2 / len(texts):.2f} ms/review")
print("Parameter storage (MiB):", round(parameter_mib(second), 1))
```

8. Write a 120-word reflection

Add a **final markdown cell** and write approximately 120 words addressing both questions:

Which checkpoint would you provisionally keep, and why? Cite your measured F1 and at least one observed error or limitation. If you tested only DistilBERT, explain what you would compare next; do not claim it outperforms an untested model.

How do processing time and F1 balance for your use-case? Cite the measured batched time and parameter-storage estimate. Explain a relevant latency, throughput, memory, truncation, or domain constraint. Do not equate ms/review with single-request latency, or parameter storage with total RAM.

Deliverables

Restart the runtime and run your notebook from top to bottom. Submit **one .ipynb file** with all required cells executed, the 250/250 class counts, timing and memory outputs, accuracy and F1, error inspection, and your 120-word reflection. Include second-model results only if you ran that optional step.

References: Amazon Polarity dataset | DistilBERT SST-2 model | Evaluate documentation